

ITM SKILLS UNIVERSITY • SCHOOL OF FUTURE TECH

Vehicle Management System

Course: Java Programming (Semester III) • Case Study 12 Final Project Documentation

Student Name: Omkar
Narvekar

Roll No:
150096725144

Degree: B.Tech Computer Science &
Engineering

1. CASE STUDY INTRODUCTION

What is the Vehicle Management System?

The **Vehicle Management System** (Vehicle Rental Management System) is a complete desktop application built in Java with a Java Swing graphical user interface. It provides a centralized, automated platform for vehicle rental business staff to manage vehicle fleets, register customers, monitor real-time vehicle availability, create bookings with automatic charge calculations, process returns, generate printable invoices, and view analytical reports.

What problem does it solve?

Traditional vehicle rental operations rely on manual register entries, paper slips, or ad-hoc spreadsheets. This causes common operational issues:

- Double-booking vehicles because availability is not tracked in real time.
- Calculation errors when computing rental charges based on variable daily rates and rental days.
- Lost rental histories and customer contact information.
- Slow search capabilities when finding vehicles by registration number or car model.
- Difficulty generating professional billing receipts and tracking fleet utilization.

This application solves all these problems by storing data in synchronized, in-memory Java collections with strict input validation and automated charge math.

What have I created?

I have created a fully functional, 100% operational Java desktop system comprising **11 modular screens** and an in-memory repository architecture without requiring external database installations. The application includes vehicle CRUD, customer CRUD, primitive boolean array tracking, HashMap lookups, TreeMap key sorting, custom exception handlers, billing, and report ledgers.

Why is it useful?

It is useful because it completely automates the rental lifecycle from customer check-in to vehicle return and billing. It provides immediate visual feedback, guarantees zero double-bookings via business validation, and serves as an academic demonstration of core Java programming concepts for B.Tech Semester III.

2. WHAT I HAVE USED

The application is built strictly using standard, platform-independent Java technologies without third-party frameworks:

Technology / Feature	Where and How It Is Used in the Project
Java (JDK 17+)	Core runtime language utilizing object-oriented programming, modern date APIs (<code>java.time.LocalDate</code>), and lambda expressions.
Java Swing GUI	Desktop interface using <code>JFrame</code> , <code>JPanel</code> , <code>CardLayout</code> , <code>GridBagLayout</code> , <code>JTable</code> , <code>JButton</code> , <code>TextField</code> , <code>JComboBox</code> , and <code>JOptionPane</code> .
Object-Oriented Programming	Clean separation into entity classes (<code>Vehicle</code> , <code>Customer</code> , <code>Rental</code> , <code>Invoice</code>) with encapsulation, constructors, and method polymorphism.
Java Collections Framework	In-memory data structures: <code>ArrayList</code> , <code>LinkedList</code> , <code>HashMap</code> , <code>TreeMap</code> , and primitive <code>boolean[]</code> array.
Exception Handling	Custom exception classes (<code>VehicleNotAvailableException</code> , <code>InvalidRentalException</code> , <code>ValidationException</code>) caught with try-catch blocks.
Data Validation	Input validation in <code>ValidationUtil.java</code> verifying phone numbers (10 digits), email patterns, positive daily rates, unique registration numbers, and valid date intervals.
CRUD Operations	Full Create, Read, Update, and Delete operations for both Vehicles and Customers with constraints (cannot delete rented vehicles).
Searching Algorithms	$O(1)$ fast search by Registration Number using <code>HashMap</code> ; linear $O(N)$ search by Model and Type using <code>ArrayList</code> .
Sorting Algorithms	Natural alphabetical order sorting using <code>TreeMap</code> ; multi-criteria sorting using <code>Comparator<Vehicle></code> (Price low/high, Model, Type).

3. PROJECT STRUCTURE

The project follows a clean, industry-standard **Model-View-Service-Repository (MVSR)** architecture with clear package separation:

```

Java CaseStudy/
├── run.sh                # One-click build and execution script
├── test.sh               # Automated integration test runner
├── README.md             # Full documentation and viva guide
├── VIVA.md               # Last-minute viva preparation guide
├── docs/screenshots/     # High-resolution application screenshots
├── src/
│   └── com/
│       └── vehiclerental/
│           ├── Main.java    # Application launcher
│           ├── model/      # Domain Entities
│           │   ├── Vehicle.java    # Fleet vehicle entity
│           │   ├── Customer.java   # Registered customer entity
│           │   ├── Rental.java     # Rental transaction entity
│           │   └── Invoice.java     # Billing receipt entity
│           ├── repository/  # Central In-Memory Storage
│           │   └── DataStore.java  # Singleton holding collections & array
│           ├── service/    # Business Logic Layer
│           │   ├── VehicleService.java    # Fleet CRUD, search, and sorting logic
│           │   ├── CustomerService.java   # Customer management & phone/email validation
│           │   ├── RentalService.java     # Booking, return lifecycle & charge calculations
│           │   ├── BillingService.java     # Invoice creation and formatting
│           │   └── ReportService.java     # Operational statistics and revenue math
│           ├── exception/  # Custom Exception Classes
│           │   ├── VehicleNotAvailableException.java # Thrown when booking rented vehicle
│           │   ├── InvalidRentalException.java      # Thrown on invalid rental dates
│           │   └── ValidationException.java         # Thrown on malformed user inputs
│           ├── util/      # Helper Utilities
│           │   ├── DateUtil.java    # Date parsing, formatting, and duration math
│           │   ├── ValidationUtil.java # Input validation logic
│           │   └── GenerateScreenshots.java # Automated UI screenshot generator
│           ├── gui/       # Swing Desktop User Interface
│           │   ├── UIStyle.java     # Shared UI color palette and styling tokens
│           │   ├── MainFrame.java   # Primary JFrame container with navigation sidebar
│           │   ├── DashboardPanel.java # Live operational KPI summary
│           │   ├── VehiclePanel.java # Vehicle CRUD interface with JTable
│           │   ├── CustomerPanel.java # Customer registration interface
│           │   ├── AvailabilityPanel.java # boolean[] availabilityStatus visualizer
│           │   ├── SearchPanel.java  # HashMap search & TreeMap sorting screen
│           │   ├── BookingPanel.java # Vehicle booking form & live charge math
│           │   ├── ReturnPanel.java  # Vehicle return processing & availability restore
│           │   ├── HistoryPanel.java # LinkedList rental history viewer
│           │   ├── BillingPanel.java  # Formatted printable receipt viewer
│           │   └── ReportPanel.java  # Revenue report and transaction ledger
│           └── test/
│               └── SystemIntegrationTest.java # 19 automated integration test cases

```

4. CODE FILE EXPLANATION

The table below explains every single Java file, its class name, and its specific purpose in the application:

File Name	Class Name	Purpose in Project
Main.java	Main	Application entry point; configures native look-and-feel and launches MainFrame on the Swing Event Dispatch Thread (EDT).
Vehicle.java	Vehicle	Entity model storing vehicle ID, registration number, brand, model, vehicle type, daily rental rate, and availability flag.
Customer.java	Customer	Entity model storing customer ID, full name, 10-digit phone, email, driving license number, and residential address.
Rental.java	Rental	Entity model representing a rental transaction with booking date, expected return date, actual return date, days, rate, and status.
Invoice.java	Invoice	Entity model that formats an official itemized text receipt containing customer, vehicle, duration, and financial charges.
DataStore.java	DataStore	Singleton repository managing the boolean[] array, ArrayList, LinkedList, HashMap, and TreeMap with pre-loaded sample data.
VehicleService.java	VehicleService	Business logic for adding/updating/deleting vehicles, HashMap lookup by registration number, and TreeMap / Comparator sorting.
CustomerService.java	CustomerService	Business logic for customer registration, update, deletion checks, and contact validation.
RentalService.java	RentalService	Handles booking reservations, duration calculations, rate math (Days × Rate), vehicle returns, and availability state toggling.
BillingService.java	BillingService	Generates Invoice instances for active or returned rentals and caches generated bills.

File Name	Class Name	Purpose in Project
ReportService.java	ReportService	Dynamically aggregates fleet KPIs: total fleet, available vehicles, rented vehicles, customer count, active/completed bookings, and gross revenue.
VehicleNotAvailableException.java	VehicleNotAvailableException	Custom checked exception thrown when an attempt is made to book a vehicle that is currently rented.
InvalidRentalException.java	InvalidRentalException	Custom checked exception thrown when return dates are prior to booking dates or date ranges are invalid.
ValidationException.java	ValidationException	Custom checked exception thrown when required fields are empty, phone numbers are not 10 digits, or duplicate IDs are entered.
DateUtil.java	DateUtil	Utility class providing date formatting (yyyy-MM-dd), parsing, and date difference calculations.
ValidationUtil.java	ValidationUtil	Utility class validating non-empty inputs, phone regex, email regex, registration number formats, and positive rental rates.
UIStyle.java	UIStyle	Standard Swing styling helper defining clean enterprise colors, fonts, button borders, and table renderers.
MainFrame.java	MainFrame	Main desktop JFrame window featuring university header banner, left navigation sidebar, CardLayout, and status bar.
DashboardPanel.java	DashboardPanel	Displays live metric stat boxes, quick navigation buttons, and recent booking activity.
VehiclePanel.java	VehiclePanel	GUI panel for adding, viewing, updating, and deleting vehicles with JTable row selection.
CustomerPanel.java	CustomerPanel	GUI panel for registering and managing customer accounts with input validation.
AvailabilityPanel.java	AvailabilityPanel	GUI panel specifically visualizing the synchronized boolean[]

File Name	Class Name	Purpose in Project
		availabilityStatus primitive array.
SearchPanel.java	SearchPanel	GUI panel demonstrating \$O(1)\$ HashMap search and multi-attribute sorting with TreeMap and Comparators.
BookingPanel.java	BookingPanel	GUI panel for creating rental reservations with live duration and charge calculations.
ReturnPanel.java	ReturnPanel	GUI panel for checking in rented vehicles, computing final dues, and restoring fleet availability.
HistoryPanel.java	HistoryPanel	GUI panel displaying all completed rentals stored in the LinkedList<Rental>.
BillingPanel.java	BillingPanel	GUI panel displaying formatted monospace billing receipts with copy-to-clipboard functionality.
ReportPanel.java	ReportPanel	GUI panel displaying complete business performance indicators and full transaction ledger.

5. OOP CONCEPTS USED

The application is designed around core Object-Oriented Programming (OOP) principles. The table below outlines how each concept is implemented in the actual codebase:

OOP Concept	Used In File	Class Name	Simple Example from Actual Project Code
Class	Vehicle.java	Vehicle	<pre>public class Vehicle { private String registrationNumber; ... } acts as blueprint.</pre>
Object	DataStore.java	DataStore	<pre>Vehicle v1 = new Vehicle("V001", "MH01AB1234", "Honda", "City", ...); instantiates a runtime object.</pre>
Encapsulation	Customer.java	Customer	All fields are declared private (e.g. <code>private String phone;</code>) and accessed only via public getters and setters (<code>getPhone()</code> , <code>setPhone()</code>).
Constructor	Rental.java	Rental	<pre>public Rental(String rentalId, Customer customer, Vehicle vehicle, ...) { this.rentalId = rentalId; ... } initializes object state.</pre>
Inheritance	ValidationException.java, MainFrame.java	ValidationException, MainFrame	<pre>public class ValidationException extends Exception inherits standard Java exception behavior; MainFrame extends JFrame inherits window capabilities.</pre>
Polymorphism	Vehicle.java, Invoice.java	Vehicle, Invoice	Method overriding: <code>@Override</code> <code>public String toString()</code> provides class-specific string representations across models.
Abstraction	<i>Not Used (Directly)</i>	<i>Not Used</i>	<i>Not directly used</i> as no custom abstract class was declared; interfaces like <code>ActionListener</code> and <code>Comparator</code> were used for behavioral contracts.

6. JAVA COLLECTIONS USED

The project fulfills the explicit requirements of Case Study 12 by utilizing multiple specialized collections and a primitive array:

Data Structure	File & Class Name	Variable Name	What It Stores and Why It Was Used
Primitive Array <code>boolean[]</code>	DataStore.java (DataStore)	boolean[] availabilityStatus	Stores availability flags indexed strictly to fleet vehicles. When vehicle at index i is rented, <code>availabilityStatus[i] = false</code> ; when returned, it is set back to true.
ArrayList <code>ArrayList<T></code>	DataStore.java (DataStore)	ArrayList<Vehicle> vehicles ArrayList<Customer> customers	Stores dynamic fleet and customer records. Provides fast $O(1)$ indexed access for populating Swing JTables.
LinkedList <code>LinkedList<T></code>	DataStore.java (DataStore)	LinkedList<Rental> rentalHistory	Stores completed rental transactions in chronological order. When a rental is closed, <code>rentalHistory.addFirst(rental)</code> inserts it at the head in $O(1)$ constant time.
HashMap <code>HashMap<K,V></code>	DataStore.java (DataStore)	HashMap<String, Vehicle> vehicleMap	Key: Registration Number (e.g. "MH01AB1234") $\rightarrow$ Value: Vehicle object. Enables instant $O(1)$ lookups without scanning an entire list.
TreeMap <code>TreeMap<K,V></code>	DataStore.java (DataStore)	TreeMap<String, Vehicle> sortedVehicles	Maintains vehicles automatically sorted in alphabetical order of registration numbers using an internal Red-Black Tree.

7. GUI COMPONENTS USED

The application uses pure Java Swing components to create a clean, responsive desktop interface. All components are styled consistently without third-party UI libraries:

Swing Component	Used In File & Class	Purpose & Application in Project
JFrame	MainFrame.java (MainFrame)	Main top-level application window containing the institutional title banner, left navigation bar, center content area, and status bar.
JPanel	All panels in com.vehiclerental.gui	Structural containers used to organize forms, metric cards, action toolbars, and table cards cleanly.
JButton	VehiclePanel, BookingPanel, etc.	Interactive trigger buttons styled with distinct visual hierarchy: Primary (Navy), Success (Green for Add/Book), Danger (Red for Delete), Secondary (Gray).
JLabel	All panels in com.vehiclerental.gui	Displays form field labels, panel headers, live KPI numeric values, and the bottom status message.
JTextField	VehiclePanel, CustomerPanel, BookingPanel	Single-line user input fields for vehicle registration numbers, customer names, phone numbers, rental dates, and daily prices.
JTable	VehiclePanel, AvailabilityPanel, etc.	Displays tabular records with custom cell heights, custom header styling, selection listeners, and color-coded status badges.
JComboBox	BookingPanel, ReturnPanel, SearchPanel	Dropdown selection menus for selecting customers, selecting available vehicles, filtering search criteria, and choosing active rentals.
JOptionPane	VehiclePanel, BookingPanel, ReturnPanel	Modal popup dialogs displaying informative success notifications, warning messages, and deletion confirmation prompts.
JScrollPane	All table and invoice panels	Wraps all JTables and the billing JTextArea to provide vertical and horizontal scrolling.
CardLayout	MainFrame.java (MainFrame)	Layout manager used in the center panel to seamlessly switch between all 10 module screens without opening separate windows.
GridBagLayout	VehiclePanel, BookingPanel, etc.	Layout manager used inside form panels to achieve professional, strictly-aligned label and textfield grids.

8. PROJECT WORKING & SYSTEM FLOW

The system executes a cohesive, end-to-end operational lifecycle:

Complete System Workflow:

Main.java → MainFrame (EDT) → DataStore Initialization → Operational Dashboard → Fleet CRUD → Customer

Registration → Availability Array Tracker → Rental Booking → Active Dispatch → Vehicle Return → Invoicing → LinkedList History Ledger → Analytical Reporting.

1. **Application Launch:** `Main.java` sets the system look-and-feel and launches `MainFrame` on the Event Dispatch Thread (EDT).
2. **Data Store Initialization:** `DataStore` initializes the fleet (5 sample vehicles), customers (3 sample customers), 1 active rental, and 1 completed rental. It synchronizes the `boolean[] availabilityStatus` array immediately.
3. **Vehicle & Customer Management:** The staff uses `VehiclePanel` and `CustomerPanel` to register vehicles and clients. Inputs are validated by `ValidationUtil`. Deleting currently rented vehicles is blocked.
4. **Availability Inspection:** The staff checks `AvailabilityPanel`, which displays the `boolean[]` array values alongside fleet rows.
5. **Vehicle Search & Sort:** The staff searches by Registration Number (instant $O(1)$ `HashMap` lookup) or sorts the fleet by daily price using `Comparator<Vehicle>`.
6. **Rental Booking:** In `BookingPanel`, the staff selects a customer and an available vehicle. The system calculates duration and charge ($\text{Days} \times \text{Rate}$). Upon confirmation, `RentalService` validates dates, marks the vehicle unavailable, updates the `boolean[]` array to false, and adds the rental to active rentals.
7. **Vehicle Return:** When the vehicle is returned, `ReturnPanel` takes the actual return date, recalculates charges if overdue, sets vehicle availability back to true, updates the `boolean[]` array to true, and moves the rental into `LinkedList<Rental> rentalHistory`.
8. **Billing & Invoicing:** In `BillingPanel`, an official text receipt is generated detailing customer info, vehicle specs, duration, and total paid dues.
9. **Executive Reporting:** `ReportPanel` aggregates real-time metrics (fleet count, active rentals, total revenue) dynamically.

9. APPLICATION SCREENSHOTS

Below are the actual high-resolution screenshots captured directly from the running Java Swing application, demonstrating every module in operation:

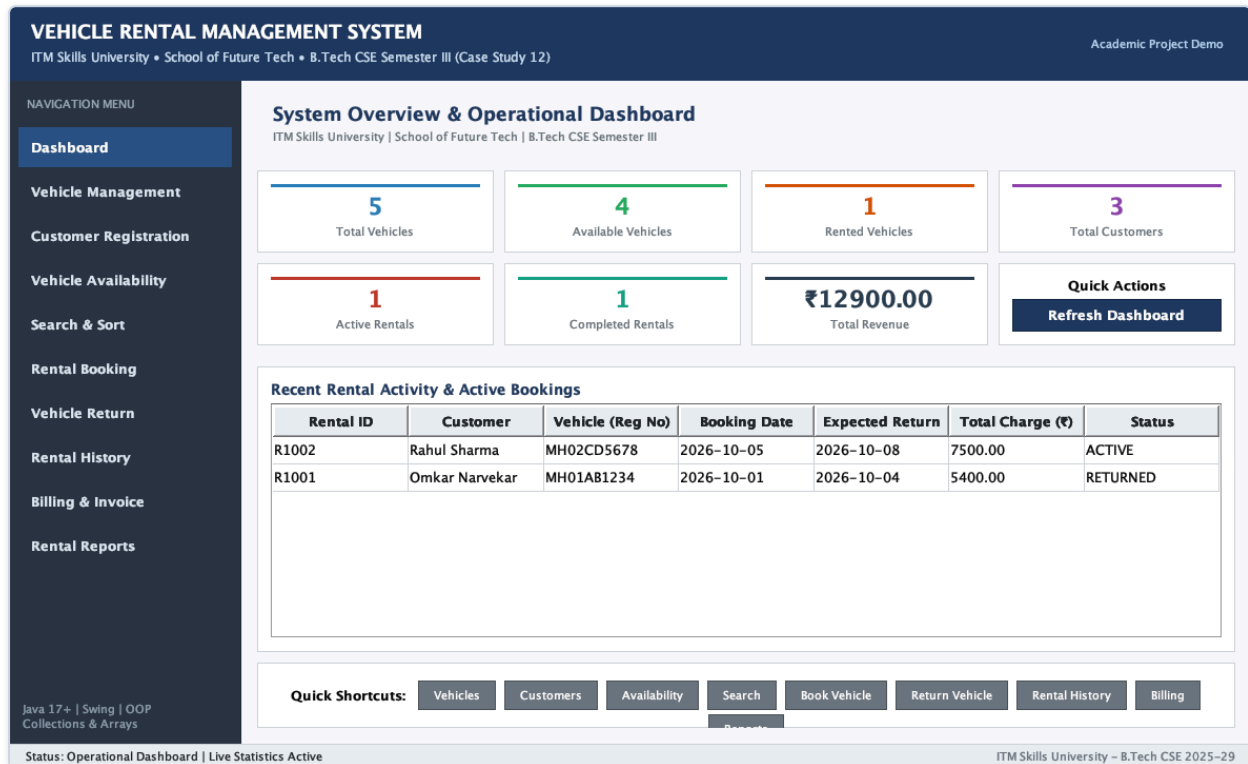


Figure 9.1: Operational Dashboard showing live fleet metrics, quick navigation shortcuts, and recent booking activity.

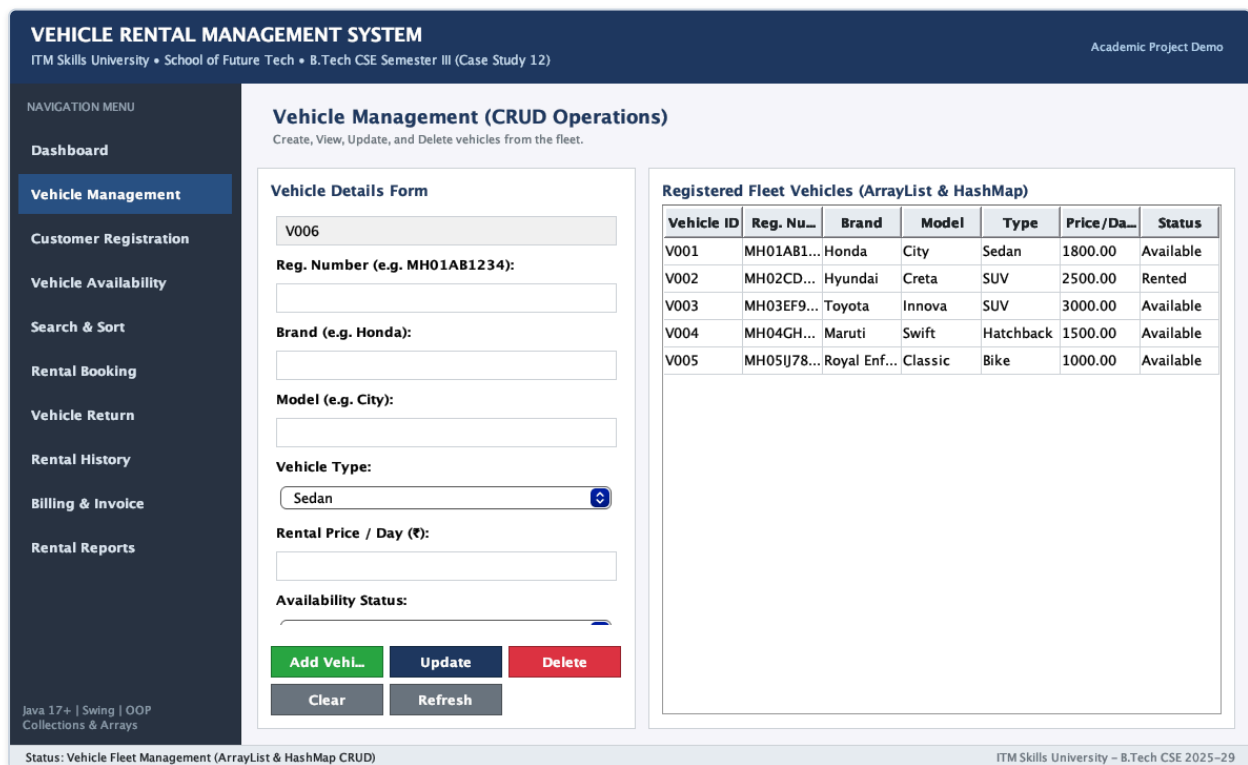


Figure 9.2: Vehicle Management screen showing the vehicle entry form and registered fleet in JTable with CRUD operations.

VEHICLE RENTAL MANAGEMENT SYSTEM

ITM Skills University • School of Future Tech • B.Tech CSE Semester III (Case Study 12)

Academic Project Demo

NAVIGATION MENU

- Dashboard
- Vehicle Management
- Customer Registration**
- Vehicle Availability
- Search & Sort
- Rental Booking
- Vehicle Return
- Rental History
- Billing & Invoice
- Rental Reports

Java 17+ | Swing | OOP
Collections & Arrays

Customer Registration & Records (ArrayList<Customer>)

Register, View, Update, and Manage customer profiles and licenses.

Customer Information Form

Customer ID:
C004

Full Name:

Phone (10 Digits):

Email Address:

Driving License No:

City / Address:

Register
Update
Delete
Clear
Refresh

Registered Customers Database

Customer L	Full Name	Phone	Email	License No	Address
C001	Omkar Nar...	9876543210	omkar.narv...	MH012022...	Dadar, Mu...
C002	Rahul Sharma	9823456789	rahul.shar...	MH022021...	Kothrud, Pu...
C003	Aditya Patil	9812345678	aditya.patil...	MH032023...	Naupada, T...

Status: Customer Registration Records (ArrayList<Customer>)

ITM Skills University – B.Tech CSE 2025–29

Figure 9.3: Customer Registration screen with validation for 10-digit phone numbers, valid emails, and driving licenses.

VEHICLE RENTAL MANAGEMENT SYSTEM

ITM Skills University • School of Future Tech • B.Tech CSE Semester III (Case Study 12)

Academic Project Demo

NAVIGATION MENU

- Dashboard
- Vehicle Management
- Customer Registration
- Vehicle Availability**
- Search & Sort
- Rental Booking
- Vehicle Return
- Rental History
- Billing & Invoice
- Rental Reports

Java 17+ | Swing | OOP
Collections & Arrays

Fleet Availability Tracker (Demonstrating: boolean[] availabilityStatus)

Tracking fleet status via synchronized Java primitive boolean array.

Filter View: All Vehicles (Array Snapshot) Refresh Status

Array Index [i]	Vehicle ID	Registration N...	Vehicle DescrL...	Type	Daily Rate (₹)	Status	Array Value (a...
[0]	V001	MH01AB1234	Honda City	Sedan	1800.00	AVAILABLE	true (Ready for ...
[1]	V002	MH02CD5678	Hyundai Creta	SUV	2500.00	RENTED	false (Rented o...
[2]	V003	MH03EF9012	Toyota Innova	SUV	3000.00	AVAILABLE	true (Ready for ...
[3]	V004	MH04GH3456	Maruti Swift	Hatchback	1500.00	AVAILABLE	true (Ready for ...
[4]	V005	MH05IJ7890	Royal Enfield C...	Bike	1000.00	AVAILABLE	true (Ready for ...

Array Size: 5 elements | Available (true): 4 | Rented (false): 1

Synchronized with DataStore.getAvailabilityStatus()

Status: Fleet Availability Tracker (boolean[] availabilityStatus Primitive Array)

ITM Skills University – B.Tech CSE 2025–29

Figure 9.4: Vehicle Availability screen displaying the synchronized boolean[] availabilityStatus primitive array.

VEHICLE RENTAL MANAGEMENT SYSTEM

ITM Skills University • School of Future Tech • B.Tech CSE Semester III (Case Study 12)

Academic Project Demo

NAVIGATION MENU

- Dashboard
- Vehicle Management
- Customer Registration
- Vehicle Availability
- Search & Sort**
- Rental Booking
- Vehicle Return
- Rental History
- Billing & Invoice
- Rental Reports

Java 17+ | Swing | OOP
Collections & Arrays

Search & Sort Vehicles (Demonstrating: HashMap, TreeMap & Comparator)

Fast O(1) HashMap lookup, TreeMap natural ordering, and Comparator sorting algorithms.

Search By: Registration Number (HashMap O(1)) Search Show All / Reset

Sort Fleet By: Registration Number (TreeMap) Apply Sort

Vehicle ID	Reg. Number	Brand	Model	Type	Price/Day (₹)	Availability
V001	MH01AB1234	Honda	City	Sedan	1800.00	AVAILABLE
V002	MH02CD5678	Hyundai	Creta	SUV	2500.00	RENTED
V003	MH03EF9012	Toyota	Innova	SUV	3000.00	AVAILABLE
V004	MH04GH3456	Maruti	Swift	Hatchback	1500.00	AVAILABLE
V005	MH05IJ7890	Royal Enfield	Classic	Bike	1000.00	AVAILABLE

Vehicles Displayed: 5

Displaying all registered fleet vehicles from ArrayList<Vehicle>

Status: Search & Sort (HashMap O(1) & TreeMap Sorting Active)

ITM Skills University – B.Tech CSE 2025–29

Figure 9.5: Search and Sort screen demonstrating O(1) HashMap lookup and Comparator / TreeMap sorting.

VEHICLE RENTAL MANAGEMENT SYSTEM

ITM Skills University • School of Future Tech • B.Tech CSE Semester III (Case Study 12)

Academic Project Demo

NAVIGATION MENU

- Dashboard
- Vehicle Management
- Customer Registration
- Vehicle Availability
- Search & Sort
- Rental Booking**
- Vehicle Return
- Rental History
- Billing & Invoice
- Rental Reports

Java 17+ | Swing | OOP
Collections & Arrays

Vehicle Rental Booking (Charge Calculation & Validation)

Create new bookings, calculate charges dynamically, and maintain active rentals.

New Booking Form

C001 – Omkar Narvekar (9876543210)

Select Available Vehicle:

MH01AB1234 – Honda City (₹1800.0/day)

Booking Date (yyyy-MM-dd):

2026-10-06

Expected Return Date (yyyy-MM-dd):

2026-10-09

Vehicle Daily Rate (₹):

1800.00

Calculated Duration:

3 Day(s)

Total Estimated Charge (₹):

Calculate Charge Confirm Booking

Clear Form Refresh Lists

Currently Active Rentals (Dispatched Vehicles)

Rental ...	Custo...	Reg. N...	Vehicl...	Bookin...	Expect...	Total C...	Status
R1002	Rahul S...	MH02C...	Hyunda...	2026-1...	2026-1...	7500.00	ACTIVE

Status: Vehicle Rental Booking & Charge Calculation

ITM Skills University – B.Tech CSE 2025–29

Figure 9.6: Rental Booking screen with automated rental duration and charge calculation (Days x Rate/Day).

VEHICLE RENTAL MANAGEMENT SYSTEM

ITM Skills University • School of Future Tech • B.Tech CSE Semester III (Case Study 12)

Academic Project Demo

NAVIGATION MENU

- Dashboard
- Vehicle Management
- Customer Registration
- Vehicle Availability
- Search & Sort
- Rental Booking
- Vehicle Return**
- Rental History
- Billing & Invoice
- Rental Reports

Java 17+ | Swing | OOP
Collections & Arrays

Process Vehicle Return (LinkedList<Rental> & Availability Restore)

Complete active rentals, calculate final billing, restore fleet availability, and update rental history.

Vehicle Return Inspection Form

Customer Name:
Rahul Sharma

Vehicle Info:
MH02CD5678 (Creta)

Original Booking Date:
2026-10-05

Expected Return Date:
2026-10-08

Daily Rate (₹):
2500.00

Actual Return Date (yyyy-MM-dd):
2026-10-06

Currently Dispatched / Active Fleet Rentals

Rental ID	Custom...	Reg. Nu...	Brand &...	Booking...	Expecte...	Total C...
R1002	Rahul Sh...	MH02C...	Hyundai...	2026-1...	2026-1...	7500.00

Status: Vehicle Return Processing (LinkedList History Maintenance)

ITM Skills University – B.Tech CSE 2025–29

Figure 9.7: Vehicle Return screen for processing returns, computing final dues, and restoring fleet availability.

VEHICLE RENTAL MANAGEMENT SYSTEM

ITM Skills University • School of Future Tech • B.Tech CSE Semester III (Case Study 12)

Academic Project Demo

NAVIGATION MENU

- Dashboard
- Vehicle Management
- Customer Registration
- Vehicle Availability
- Search & Sort
- Rental Booking
- Vehicle Return
- Rental History**
- Billing & Invoice
- Rental Reports

Java 17+ | Swing | OOP
Collections & Arrays

Completed Rental History (Demonstrating: LinkedList<Rental>)

Maintaining closed transactions in chronological order using Java Collections LinkedList.

Rental ID	Customer ...	Customer ...	Vehicle Re...	Model	Booking D...	Actual Ret...	Days	Total Amo...	Status
R1001	Omkar Narv...	9876543210	MH01AB1234	Honda City	2026-10-01	2026-10-04	3	5400.00	RETURNED

Completed Rentals in LinkedList: 1 | Total Completed Revenue: ₹5400.00

Data Structure: java.util.LinkedList<Rental>

Status: Completed Rental History (LinkedList<Rental>)

ITM Skills University – B.Tech CSE 2025–29

Figure 9.8: Rental History screen displaying completed transactions maintained in a LinkedList<Rental>.

VEHICLE RENTAL MANAGEMENT SYSTEM

ITM Skills University • School of Future Tech • B.Tech CSE Semester III (Case Study 12)

Academic Project Demo

NAVIGATION MENU

- Dashboard
- Vehicle Management
- Customer Registration
- Vehicle Availability
- Search & Sort
- Rental Booking
- Vehicle Return
- Rental History
- Billing & Invoice**
- Rental Reports

Java 17+ | Swing | OOP
Collections & Arrays

Billing & Invoice Generation

Generate official itemized rental invoices and receipts for customers.

Select Transaction: R1002 – Rahul Sharma (MH02CD5678) [A... OR Rental ID: Generate Bill

Invoice Receipt Preview

=====

VEHICLE RENTAL MANAGEMENT SYSTEM
ITM Skills University – Final Project
OFFICIAL INVOICE

=====

Invoice No : INV-0101 Date : 2026-10-06
Rental ID : R1002 Status : ACTIVE / PENDING

=====

CUSTOMER DETAILS:
Customer ID : C002
Name : Rahul Sharma
Phone : 9823456789
Email : rahul.sharma@gmail.com
DL Number : MH0220210098765

=====

VEHICLE DETAILS:
Reg. Number : MH02CD5678
Vehicle : Hyundai Creta
Type : SUV
Daily Rate : ₹2500.00

=====

RENTAL DURATION & CHARGES:
Booking Date : 2026-10-05
Expected Return : 2026-10-08
Duration (Days) : 3 day(s)
Rate Per Day : ₹2500.00

=====

Status: Customer Billing & Invoice Generator

ITM Skills University – B.Tech CSE 2025-29

Figure 9.9: Official Billing and Invoice screen generating formatted itemized receipts with payment status.

VEHICLE RENTAL MANAGEMENT SYSTEM

ITM Skills University • School of Future Tech • B.Tech CSE Semester III (Case Study 12)

Academic Project Demo

NAVIGATION MENU

- Dashboard
- Vehicle Management
- Customer Registration
- Vehicle Availability
- Search & Sort
- Rental Booking
- Vehicle Return
- Rental History
- Billing & Invoice
- Rental Reports**

Java 17+ | Swing | OOP
Collections & Arrays

Fleet Operations & Rental Summary Report

Aggregated financial statistics, vehicle utilization metrics, and complete transaction log.

Key Operational Performance Indicators Refresh Report

5 Total Fleet	4 Available	1 Rented	3 Customers	1 Active	1 Completed	₹12900.00 Revenue
------------------	----------------	-------------	----------------	-------------	----------------	----------------------

All Rental Transactions Ledger (Active & Completed)

Rental ID	Customer	Registration...	Vehicle	Booking Date	Return Date	Duration	Total Amou...	Status
R1002	Rahul Sharma	MH02CD5678	Hyundai Creta	2026-10-05	2026-10-08 ...	3 Day(s)	7500.00	ACTIVE
R1001	Omkar Narve...	MH01AB1234	Honda City	2026-10-01	2026-10-04	3 Day(s)	5400.00	RETURNED

Status: Executive Fleet Operations & Revenue Report

ITM Skills University – B.Tech CSE 2025-29

Figure 9.10: Analytical Reports screen showing real-time KPIs and comprehensive transaction ledger.

10. CONCLUSION

What I Created

For my B.Tech Computer Science Semester III Java Programming final project, I successfully developed a complete, fully functional **Vehicle Rental Management System** adhering strictly to the requirements of Case Study 12. The application provides an integrated desktop platform that manages vehicles, registered customers, reservations, returns, real-time availability tracking, charge calculations, invoicing, and reporting without requiring any external database software.

Java Concepts Used

Through this project, I demonstrated the practical implementation of essential Java concepts:

- **Object-Oriented Programming (OOP):** Modeled real-world entities using encapsulated classes (Vehicle, Customer, Rental, Invoice) with constructors, getters, setters, and method overriding.
- **Java Collections Framework:** Successfully implemented and synchronized an ArrayList for dynamic records, a LinkedList for chronological rental history, a HashMap for instant $O(1)$ vehicle search by registration number, and a TreeMap for natural key sorting.
- **Primitive Array:** Genuinely utilized a boolean[] availabilityStatus array to track vehicle availability by index across the application.
- **Exception Handling & Validation:** Implemented robust custom exceptions (VehicleNotAvailableException, InvalidRentalException, ValidationException) and comprehensive regex validation to safeguard data integrity.
- **Java Swing GUI:** Constructed an intuitive desktop user interface utilizing JFrame, CardLayout, GridBagLayout, JTable, and JOptionPane.

What I Learned

This case study gave me hands-on experience in architecting a multi-tiered software application using the Model-View-Service-Repository pattern. I learned how to choose the right data structure for specific performance requirements (e.g., HashMap for fast lookups vs. TreeMap for sorted views), how to manage application state across multiple GUI panels, and how to write clean, maintainable, and viva-ready Java code.

Student Name: Omkar Narvekar

Roll No: 150096725144

Institution: ITM Skills University (B.Tech CSE)